URBANVERSE

TEAM INTEGRATION GUIDE

Project: City-Scale Digital Twin for Multi-Hazard Prediction and Urban Resilience Planning

Hackathon target: Build a convincing, working flood-risk MVP for a small pilot area in Mangalore. Use real map/geographical data where practical and clearly label simulated rainfall and risk outputs.

1. What we are building

UrbanVerse is a small-area digital twin prototype. The demo should let a user view a real map area, adjust a simulated rainfall level, see a flood-risk layer, inspect affected zones and receive basic response suggestions. It is a prototype, not an operational flood-warning system.

MVP acceptance checklist

- A real map of one small, clearly named pilot area in Mangalore.
- A working map/dashboard with a visible selected area.
- A rainfall control that changes the simulation input.
- A risk score and Low / Medium / High label for zones.
- A heatmap or coloured risk overlay on the map.
- A short list of affected zones and response suggestions.
- A visible note: 'Simulation only; not a live emergency warning.'

2. Shared architecture

Keep the first version simple. Do not add services or technologies just because an AI tool suggests them.

Part	Responsibility	Owner
Frontend	React dashboard, map, controls, results	Member 2
GIS data	Pilot-area GeoJSON, zones, roads/water/terrain if available	Member 3
Simulation	Rainfall input, zone risk score, affected zones	Member 4
Backend	FastAPI endpoints; combines simulation + GIS; serves results	Member 1 / You

3. Repository structure

Agree on this structure before generating files. If your existing repository already has a structure, adapt it once and tell everyone; do not create competing structures.

```
UrbanVerse/
■■■ frontend/ # Member 2: React app
■■■ backend/ # Member 1: FastAPI app
■ ■■■ app/ # API routes and integration
■ ■■■ requirements.txt
■■■ data/ # Member 3: GeoJSON and data notes
■■■ simulation/ # Member 4: risk calculation module
■■■ docs/ # setup, API contract, sources
■■■ README.md
```

4. The interface contract: agree before coding

The frontend should call one backend endpoint for the simulation. The backend should call the simulation module and return a consistent response. Do not let each person invent different field names.

Request example

```
POST /api/simulate
{
  "rainfall_mm": 80,
  "area_id": "pilot-area-01"
}
```

Response example

```
{
  "area_id": "pilot-area-01",
  "rainfall_mm": 80,
  "zones": [
    {"zone_id":"zone-a","risk_score":82,"risk_level":"High"},
    {"zone_id":"zone-b","risk_score":46,"risk_level":"Medium"}
  ],
  "affected_zones": ["zone-a"],
  "recommendations": ["Prioritise checks near low-lying roads"],
  "simulation_only": true
}
```

Minimum contract rules

- risk_score is a number from 0 to 100.
- risk_level is exactly Low, Medium or High.
- zone_id must match an ID in the GIS data.
- rainfall_mm is a numeric simulation input, not a live measurement.
- simulation_only must remain true in the hackathon prototype.
- If a field changes, tell all teammates before changing code.

5. GitHub collaboration rules

1. Keep main as the working, integrated version. Do not experiment directly on main.
2. Create one branch per person: backend, frontend, gis-data, simulation.
3. Each person edits only their assigned folders unless the team agrees otherwise.
4. Before starting: pull the latest main and update your branch.
5. Commit small working changes with clear messages.
6. Push your branch and open a Pull Request. Do not merge your own changes without checking the app.
7. Only one person (Member 1) coordinates integration into main. Member 2 helps test the complete UI.
8. Never have two AI tools or two people editing the same file at the same time.

6. How the modules merge

1. Member 3 supplies a small GeoJSON file with stable zone IDs, for example zone-a and zone-b.
2. Member 4 writes a simulation function that accepts rainfall_mm and those zone IDs, then returns risk scores.
3. Member 1 wraps that function in POST /api/simulate and validates the response.
4. Member 2 builds the UI against the agreed request/response example, initially using mock data if the backend is not ready.
5. Connect the real endpoint, test the same rainfall input, and verify the heatmap colours match the returned risk levels.

7. 24-hour build order

- **Hours 0–2:** Confirm pilot area, repository structure, API contract and tool setup.
- **Hours 2–6:** Map appears; backend starts; GIS data is prepared; simulation returns sample scores.
- **Hours 6–12:** Connect map, rainfall control and simulation endpoint.
- **Hours 12–17:** Add heatmap, affected zones and recommendations.
- **Hours 17–21:** Integration testing, bug fixes and fallback demo data.
- **Hours 21–24:** Freeze features, rehearse demo and prepare presentation.

8. Definition of done

A task is not done because an AI tool generated code. It is done when the owner runs it, tests the expected input/output, commits it, and another teammate can run it using the README instructions.

Scope warning

Do one flood scenario well. Do not attempt scientifically accurate city-wide multi-hazard prediction, live sensor ingestion, routing optimisation and a full 3D city model in 24 hours. Label assumptions and simulated values honestly.